

Design Decisions Documentation

Overview

This document explains the key design decisions made during the development of the Axiom Trade replica. Each decision was made to achieve pixel-perfect accuracy, optimal performance, maintainable code, and excellent user experience.

1. Architecture and Framework Decisions

1.1 Next.js 14 App Router

Decision: Use Next.js 14 with App Router instead of Pages Router.

Rationale:

The App Router provides a modern architecture with improved data fetching patterns and layout composition. Server Components enable server-side rendering for better performance. Route Groups improve organization of routes and layouts. It is the recommended and future-proof approach for new Next.js projects and offers better TypeScript integration.

Trade-offs:

There is a slightly steeper learning curve compared to the Pages Router, and some libraries may not yet be fully compatible, which is mitigated by choosing compatible alternatives.

1.2 TypeScript Strict Mode

Decision: Enable TypeScript strict mode with comprehensive type definitions.

Rationale:

Strict mode improves type safety by catching errors at compile time, enhances IDE support, serves as self-documenting code, enables safer refactoring, and improves team collaboration through clear contracts.

Implementation:

All components are fully typed. Custom types are defined in the types directory. Any type usage is avoided except where absolutely necessary, and strict null checks are enabled.

1.3 Atomic Design Architecture

Decision: Organize components using Atomic Design principles (Atoms, Molecules, Organisms).

Rationale:

This structure improves reusability, maintainability, scalability, testing, and collaboration by creating a clear component hierarchy and separation of concerns.

Benefits:

Clear separation of concerns, easier modification, component reuse, and adherence to industry best practices.

2. State Management Decisions

2.1 Redux Toolkit for Global State

Decision: Use Redux Toolkit for managing global application state such as token lists, sorting, and selected tokens.

Rationale:

Redux Toolkit provides predictable state updates, time-travel debugging via DevTools, scalability, strong TypeScript support, and efficient performance with proper selectors.

Implementation:

A single store with a token slice is used along with typed hooks and immutable updates via Immer.

When to Use:

Global state shared across components, complex logic, and state persisting across navigation.

2.2 React Query for Server State

Decision: Use React Query for data fetching and caching.

Rationale:

It offers automatic caching, background refetching, built-in loading and error handling, optimistic updates, and request deduplication.

Configuration:

A 60-second stale time is set for token data, refetch on window focus is disabled, and automatic retry is enabled on failure.

Benefits:

Clear separation of server and client state, reduced boilerplate, improved error handling, and automatic cache invalidation.

2.3 Local State for UI

Decision: Use React useState for component-level UI state.

Rationale:

This keeps state simple, avoids unnecessary re-renders, improves encapsulation, and reduces boilerplate.

Examples include modal visibility, dropdown state, form inputs, and temporary UI flags.

3. Styling Decisions

3.1 Tailwind CSS

Decision: Use Tailwind CSS for styling instead of CSS-in-JS or traditional CSS.

Rationale:

Tailwind provides utility-first styling, consistency, performance optimization, responsive design, dark mode support, and improved maintainability.

Custom Configuration:

Extended color palette, custom animations, consistent border radius system, and custom scrollbar styling.

3.2 Dark Theme Design

Decision: Implement a dark theme matching Axiom Trade's design.

Rationale:

Dark themes allow for pixel-perfect design matching, reduce eye strain, and provide a modern fintech aesthetic.

Implementation:

CSS variables and HSL color systems ensure consistency and easy theme adjustments.

Color Palette:

Background: Dark blue-gray (222 47% 11%)

Foreground: White (0 0% 100%)

Primary: Blue (217 91% 60%)

Success: Green (142 76% 36%)

Destructive: Red (0 84% 60%)

3.3 shadcn/ui Component Library

Decision: Use shadcn/ui for base UI components.

Rationale:

This library provides accessibility, full customization, strong TypeScript support, Tailwind integration, and no vendor lock-in.

Components used include Button, Badge, Table, Dialog, Popover, and Tooltip.

4. Performance Optimizations

4.1 React.memo for Component Memoization

Decision: Wrap frequently re-rendered components with React.memo.

Rationale:

This prevents unnecessary re-renders and ensures smooth performance during real-time updates.

Memoized components include TokenRow, RealTradingChart, TradingPanel, and AxiomTokenTable.

4.2 useMemo and useCallback

Decision: Use useMemo and useCallback for expensive calculations and stable references.

Examples include memoized filtered token lists, sort handlers, and chart data generation.

4.3 Fixed Dimensions to Prevent Layout Shifts

Decision: Use fixed heights and widths where necessary to reduce cumulative layout shift and improve user experience.

4.4 Optimized Bundle Size

Decision: Use selective imports, tree-shaking, dynamic imports, and SWC minification to reduce bundle size and improve load times.

5. Data Fetching and Real-Time Updates

5.1 WebSocket Mock for Real-Time Updates

Decision: Use setInterval to simulate WebSocket updates.

Rationale:

This allows demonstration of real-time features in development without backend dependencies.

5.2 Mock Data Structure

Decision: Use comprehensive mock data reflecting real API structure to enable easy migration later.

6. User Experience Decisions

6.1 Loading States

Decision: Implement skeletons, shimmer effects, and progressive loading to provide user feedback and improve perceived performance.

6.2 Error Handling

Decision: Use error boundaries and fallback UI for graceful recovery and debugging.

6.3 Animations and Transitions

Decision: Use CSS transitions for visual feedback and polished interactions.

7. Responsive Design Decisions

7.1 Mobile-First Approach

Design starts for small screens and scales up for tablets and desktops.

7.2 Horizontal Scrolling for Tables

Enables full data visibility on mobile without restructuring the table.

7.3 Responsive Chart

Ensures usability across devices with flexible layouts and minimum height constraints.

8. Component-Specific Decisions

8.1 TradingView Lightweight Charts

Used for performance, feature-rich charts, and industry-standard visualization.

8.2 Token Detail Page Layout

Flexbox layout prioritizes chart visibility with a dedicated trading panel sidebar.

8.3 Expandable Bottom Controls

Keeps UI clean while providing advanced options when needed.

9. Code Quality Decisions

9.1 DRY Principles

Reusable utilities and shared components reduce duplication and improve consistency.

9.2 Comprehensive Type Definitions

All types are explicitly defined to improve safety and documentation.

9.3 Error Boundaries

Used to isolate and manage errors without crashing the entire application.

10. Deployment Decisions

10.1 Vercel Platform

Chosen for optimized Next.js performance, easy deployment, and global edge delivery.

10.2 Environment Configuration

Mock data is used initially with scope for future environment variable integration.

11. Testing and Quality Assurance

11.1 TypeScript as First Line of Defense

Used for compile-time safety and documentation.

11.2 ESLint Configuration

Based on Next.js recommended rules for consistency and best practices.

12. Future Considerations

12.1 Real API Integration

Mock services can be replaced with real APIs following the same structure and logic.

12.2 Additional Features

Potential additions include authentication, wallet integration, real trading, advanced analytics, and notifications.

Summary

These design decisions ensure pixel-perfect accuracy, high performance, maintainability, scalability, and excellent user experience. The chosen architecture and practices support future growth and adaptability as project requirements evolve.